Recursive analysis and data structure review

William Hendrix

Lecture 3

Outline

- Iterative analysis review
- Recursive analysis
- The Master Theorem
- Stacks, queues

Review: iterative algorithm analysis

- Identify loops and function calls
  - Everything else is *Θ(1)*
- *For loops*:
  - Estimate number of iterations
    - Incrementing by *c*: divide range by *c* to get iterations
    - Multiplying by *c:* take $\log_c$ of the end/start ratio
  - Estimate loop body running time
    - Might depend on iteration #
  - If iterations don't depend on i: # iterations * time per iteration
  - Otherwise: sum up all iterations

$$\sum_{i=1}^{x} i = \Theta(x^2) \qquad \sum_{i=1}^{x} \frac{1}{i} = \Theta(\lg x) \qquad \sum_{i=1}^{x} 2^i = \Theta(2^x) \qquad \sum_{i=1}^{x} \frac{1}{2^i} = \Theta(1)$$

- *For functions:*
  - Analyze other functions separately
  - Recursive functions: set up a recurrence and solve
- Overall complexity: largest loop or function call complexity

Recursive analysis

- **Challenge:** recursive functions call themselves repeatedly (down to base case)
- **Main idea:** use recurrence to define runtime recursively

- **Example:** factorial
  - *T(n)*: time to compute $n!$

```
Input: n: number to calculate factorial
Output: n!
Algorithm: Factorial
if n = 0 then
    return 1
else
    temp = Factorial(n − 1)
    return n ∗ temp
end
```

Recursion tree analysis

- Technique to analyze algorithm complexity of recursive algorithms
- **Main idea**
  - Sketch a tree that represents all recursive calls made
    - Start with n, split according to size and # of recursive calls
  - Add up complexity on each level of tree
    - Apply nonrecursive complexity to every node
    - Often helpful to estimate tree height
  - Add up the total complexity of all the nodes
- **Example**
  - $T(n) = T(n-1) + \Theta(n)$

Recursion tree example

- Identify the complexity class for T(n) when

$$T(n) = 2T(n-1) + \Theta(1),\ T(1) = \Theta(1)$$

2 rec. calls of size n-1

Everything else in function

Recursive analysis exercise

```
Input: data: array of n integer
Input: n: size of data
Input: t: target value to search for
Output: index i such that data[i] = t, or 0 if t ∉ data
Algorithm: BinSearch
if n = 1 then
    return whether data[1] = t
else
    mid = ⌈n/2⌉
    if data[mid] = t then
        return mid
    else if data[mid] > t then
        return BinSearch(data[1..mid], t)
    else
        return BinSearch(data[mid + 1..n], t)
    end
end
```

1. Give a recurrence that describes the worst-case time complexity of BinSearch. (*Hint*: how many recursive calls will you make?)
2. Draw a recursion tree for your recurrence.
3. Solve the recurrence.

The Master Theorem

- Many recursive algorithms have complexity of the form:

  $T(n) = aT(n/b) + f(n)$

  - n/b : size of recursive calls
  - a : number of recursive calls (often $a = b$)
  - $f(n)$: time required for other code
- **Master Theorem**
  - Gives complexity for $T(n)$ based on a, b, and $f(n)$

  1. Calculate $c = log_b(a)$
  2. Compare complexity of $f(n)$ to n^c
     - If $f(n) = \Theta(n^c)$, $T(n) = \Theta(f(n)\lg n)$
     - Otherwise, if $f(n)$ is *strictly smaller* than $O(n^c)$, $T(n) = \Theta(n^c)$
       - $f(n) = O(n^{c-e})$, for some $e > 0$
     - Otherwise, if $f(n) = \Omega(n^{c+e})$ <u>and</u> f is *regular*, $T(n) = \Theta(f(n))$
       - <u>Strictly more</u> than n^c
       - Regular: $af(n/b) < f(n)$, for large n

Formal statement of Master Theorem

Master Theorem. If T is an increasing function that satisfies the recurrence

$$T(n) = aT(n/b) + f(n)$$

where $a \geq 1$ and $b \geq 1$, then:

$$T(n) = \begin{cases} \Theta(n^c), & \text{if } f(n) = O(n^{c-\epsilon}) \text{ for some } \epsilon > 0 \\ \Theta(n^c \lg n), & \text{if } f(n) = \Theta(n^c) \\ \Theta(f(n)), & \text{if } f(n) = \Omega(n^{c+\epsilon}) \text{ for some } \epsilon > 0 \\ & \quad \underline{\text{and}}\ af(n/b) < f(n) \text{ for large } n \end{cases},$$

where $c = \log_b(a)$.

Almost: $T(n) = \Theta(n^c + f(n))$ unless n^c and $f(n)$ are same size

For purposes of the Master Theorem, you may ignore floor and ceiling

E.g., $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n)$

$$= 2T(n/2) + \Theta(n)$$

Warning: cases 1+3 must be polynomially different (<u>not log</u>)

Master Theorem application

Master Theorem. If $T(n) = aT(n/b) + f(n)$,

$$T(n) = \begin{cases} \Theta(n^c), & \text{if } f(n) = O(n^{c-\epsilon}) \text{ for some } \epsilon > 0 \\ \Theta(n^c \lg n), & \text{if } f(n) = \Theta(n^c) \\ \Theta(f(n)), & \text{if } f(n) = \Omega(n^{c+\epsilon}) \text{ for some } \epsilon > 0 \\ & \underline{\text{and}}\ af(n/b) < f(n) \text{ for large } n \end{cases}$$

- *$T(n) = T(n/2) + \Theta(1)$*

1. Identify variables
2. Calculate *c*
3. Decide case
4. Report complexity (test regularity if case 3)

Master Theorem exercises

- What is c for the following recurrences?
- What case does $f(n)$ fall under?
- What is the asymptotic complexity for the following recurrences?
  Write "n/a" if the Master Theorem does not apply.

1. $T(n) = 2T(n/2) + \Theta(1)$
2. $U(n) = 4U(n/2) + \Theta(n^2)$
3. $V(n) = 9V(n/9) + \Theta(n\lg n)$
4. $W(n) = 3W(n/3) + \Theta(\lg n)$
5. $X(n) = 2X(n/4) + \Theta(n\lg n)$
6. $Y(n) = 3Y(n/9) + \Theta(1)$
7. $Z(n) = Z(n/2) + \Theta(n)$

	c	**Case**	**Complexity**
$T(n)$			
$U(n)$			
$V(n)$			
$W(n)$			
$X(n)$			
$Y(n)$			
$Z(n)$			

Data structures in memory

- Contiguous
  - Allocate single chunk of memory
  - Retrieve elements by locating data's index in chunk
  - Very fast if elements are accessed consecutively (caching)
  - No memory wasted on storing pointers
  - Following *k* links takes *O(k)* time vs. *O(1)* for pointer arithmetic
- Link-based
  - Data are stored in small "islands" connected via pointers
  - Retrieve elements by starting at the head/tail/root and traversing links
  - Supports data with irregular structure (graphs, trees)
  - Easier for memory manager to allocate
  - Easy to modify structure by changing links (vs. copying data)
- Hybrid

Abstract data types

- Abstract data type
  - Specifies *behavior* of data structure, not *implementation*
  - Similar to interface

- Stack
  - 3 main operations
  - push(x)
    - Add a new element to the top of the stack
  - pop()
    - Remove the element on the top of the stack
  - peek()
    - Return the top of stack without removing
  - Analogy: stack of plates
    - First thing removed was last thing added
    - Last-In, First-Out (LIFO) order

Stack implementations

- Can be implemented using linked list or array
  - Array implementation generally preferred

<u>General strategy</u>

- *capacity*: allocated array size
- *size:* number of elements/used capacity
- Stack: add/remove from end

- Pop is straightforward
  - Decrement *size*
- Increase *size* when adding elements
  - Need to increase *capacity* when array is full

Array enlargement policy

- **Bad policy:** increment by 1
  - Cost: $\Theta(n)$ every time
  - n pushes: $\Theta(1 + 2 + \ldots + n)) = \Theta(n^2)$
  - *Average cost per push:* $O(n)$
- **Better policy:** increment by k (k=10, 100, etc.)
  - Cost: $\Theta(n)$ every k^{th} time
  - n pushes: $\Theta(1 + (1 + k) + \ldots + (1 + k\lfloor \frac{n}{k} \rfloor)) = \Theta\left(\frac{n^2}{k}\right)$
  - *Average cost per push:* $\Theta(\frac{n}{k})$
  - *Caution*: space trade-off
    - Increment by 1B: small stacks will be mostly empty space
- **Best policy:** double array size
  - Cost: $\Theta(n)$ after powers of two
  - n pushes: $\Theta(1 + 2 + 4 + \ldots + 2^{\lfloor \lg n \rfloor}) = \Theta(n)$
  - *Average cost per push:* $\Theta(1)$
  - Array will be at least half-full if not deleting

Amortized analysis

Queues

- Data structures related to stacks
- Three main operations
  - enqueue(x)
    - Add a new element to the back of the queue
  - dequeue()
    - Remove the element to the front of the queue
  - peek()
    - Return the front of queue without removing
  - Analogy: line
    - First-In, First-Out (FIFO) ordering
    - Complementary to stack

- Deque ("deck")
  - Double-Ended QUEue
  - 4 operations: push_front(), push_back(), pop_front(), pop_back()
  - Can simulate stack (push_back/pop_back) or queue (push_back/pop_front)

Stack/queue applications

- Stacks more common
  - Reversing a string
  - Matching parentheses
  - Undo
    - Pop most recent modification to undo
  - *Backtracking* (later)

- Queues (and dequeues) less common
  - Resource allocation
    - E.g., buffering, printer queues
    - Everyone waits roughly the same amount of time
  - Also used in graph and tree traversals

Operation analysis

- **push(x)**
  - Depends on *capacity*
    - Allocated memory
  - If capacity > size, just add
  - Otherwise, enlarge first
  - *Θ(1) time, amortized*
- **pop()**
  - Remove last element
  - *Θ(1) time*
- **enqueue(x)**
  - Similar to push()
  - *Θ(1) time, amortized*
- **dequeue(x)**
  - Advance "head" pointer
  - *Θ(1) time*

```
Algorithm: Push(x)
if n ≥ c then
    Enlarge()
end
n = n+1
stack[n] = x
```

```
Algorithm: Enlarge()
c = 2c
newstack = allocate(c)
for i = 1 to n do
    newstack[i] = stack[i]
end
free stack
stack = newstack
```

```
Algorithm: Pop()
if n = 0 then
    error "Stack is empty"
end
n = n - 1
return data[n]
```

Stack/queue summary

- Data structures that access newest/oldest element added
  - Array implementations generally preferred
  - Linked list possible

Operation	Stack	Queue	Deque
Push(x) / Enqueue(x)	$\Theta(1)$*	$\Theta(1)$*	$\Theta(1)$*
Pop() / Dequeue()	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
Peek()	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$

* amortized time

- All operations are constant amortized time
- Do not support random access
- Choice of stack/queue is based on desired access order
  - Stack: Last In, First Out (LIFO)
  - Queue: First In, First Out (FIFO)
  - Deque: Can simulate either

Stack/queue exercise

- In postfix expressions, operators appear after the values they affect
  - E.g., 5 7 + for 5 + 7 = 12
- Operators combine last two numbers into a value
  - E.g., 6 1 - 7 + = 5 7 + = 12
  - E.g., 6 1 7 - + = 6 -6 + = 0
- Write pseudocode for an algorithm that accepts a mixed array of operators (+, -, *, /) and numbers that evaluates the expression

```
In:  op:  mixed array of numbers and operators representing a postfix expression
In:  n:  size of op
Out:  value of op as a postfix expression
Eval(op):
```

Coming up

- Stacks, queues, deques
- BSTs
- Hash tables
- Sets, Maps
- Priority queues
- Union-Find

- **Recommended readings:** Chapter 2 (all except 2.2.1), sections 4.1, 4.2, 6.1, 6.2, and 6.3
  - Consider reading section 6.4
  - We won't cover cuckoo hashing, but it is a powerful idea
  - *Practice problems:* R-2.2, R-2.5, C-2.3, C-2.13, A-2.2, R-4.1, R-4.6, C-4.5, C-4.7, A-4.1, A-4.2, R-6.1, R-6.2, R-6.4 to R-6.7, C-6.6, A-6.1, A-6.6